

UMass Marketplace: Data Models and Module Diagrams

Team 10

Manager: Vidhita Mittal

Members: Vidhi Dave, Ngan Nguyen,
Gayatri Marathe, Arkar Myint

Table of contents

1. Frameworks

3. Function signatures

2. End point Routes

4. Testing Functions

Frameworks

Frameworks

- Front end

- UI/Prototyping: **Figma** for designing screens + iterating on user flows
- **TS + JS** for client side logic and interactivity
- **HTML/CSS** for structural layout + UI styling

- Backend/API

- **Express.js** (routing + controllers for listings, search/filter, seller profiles, messaging)

Frameworks

- ORM

- **TypeORM** (map database tables to code models)

- DBMS

- **PostgreSQL** to store marketplace data + relationships

Endpoint Routes

API Routing

HTTP Method + URL Path = API Endpoint

e.g. GET `http://umassmarketplace:3000/api/listings/1025`

- URL (the address) and the HTTP Method (the action) work together to form a route.

HTTP Method + URL Path + Query Parameters = Filtered Request to an API Endpoint

e.g. GET `http://umassmarketplace:3000/api/listings?q=chair&maxPrice=100&status=active`

- Query parameters filter or modify the results
- Common for search or browse endpoints

Core Resource: /products

An example to show how a single resource (Products) handles the full life cycle of data (CRUD: Create, Read, Update, Delete)

Method	Endpoint	Description
POST	/products	Create a new product record
GET	/products	Retrieve all products
GET	/products/:id	Retrieve specific product details
PATCH	/products/:id	Update specific fields of a product
DELETE	/products/:id	Remove a product record

:id is a placeholder for real data (e.g. GET /products/1025)

Core Resource: /listings

Method	Endpoint	Description
POST	/listings	Create a new listing record
GET	/listings	Retrieve all products
GET	/listings/:id	Retrieve specific product details
GET	/listings/:id/?q=&category=&...	Retrieve specific fields of a product record with query parameters
PATCH	/listings/:id	Update specific fields of a product (title, price, status, etc.)
PATCH	/listings/:id/status	Update status of listings (sold/active/etc)
DELETE	/products/:id	Remove a listing record

Function Signatures

What is a Function Signature?

A function signature includes:

- **Function name**
- **Input parameters**
- **Return type**

Example:

createListing(data: ListingInput, userId: string): Promise<Listing>

Layer	Example Signature
Controller	createListing(req: Request, res: Response): Promise<void>
Service	createListing(data: ListingInput, userId: string): Promise<Listing>
Model	ListingModel.create(data): Promise<Listing>

Controllers receive HTTP requests, services handle the business logic, and models interact with the database.

Input / Output Design

`getListingById(id: string): Promise<Listing | null>`

Input:

- id: string

Output:

- Listing if found
- null if not found

Request Flow

GET /listings/:id



listingController.getListingById(req,res)



listingService.getListingById(id)



ListingModel.findById(id)

Main Marketplace Functions

Function Signature	Description
register(data: RegisterInput): Promise<AuthResponse>	Creates a new user account and returns authentication token
login(data: LoginInput): Promise<AuthResponse>	Authenticates a user and returns login token
createListing(data: ListingInput, userId: string): Promise<Listing>	Creates a new marketplace listing
getListings(query: ListingQuery): Promise<Listing[]>	Retrieves listings with optional filters
getListingById(id: string): Promise<Listing null>	Returns details of a specific listing
updateListing(id: string, data: ListingInput): Promise<Listing null>	Updates listing information
deleteListing(id: string): Promise<void>	Removes a listing from the marketplace

Supporting System Functions

Function Signature	Description
createProduct(data: ProductInput): Promise<Product>	Creates product record associated with listings
getProducts(): Promise<Product[]>	Retrieves all products
createCategory(data: CategoryInput): Promise<Category>	Creates a marketplace category
getCategories(): Promise<Category[]>	Returns all listing categories
createSeller(data: SellerInput): Promise<Seller>	Creates seller profile
getSellerById(id: string): Promise<Seller null>	Retrieves seller information

Testing Functions

Verify that core marketplace features work correctly before deployment.

We will test:

- Authentication
- Listings
- Search/filter
- Favorites
- Messaging
- Reports/Blocks

Testing approach:

- Unit test backend functions
- Test API endpoints with sample requests
- Check returned JSON / UI output
- Verify database changes in Postgres/pgAdmin after create, update, or delete actions

Component	Details
Feature Being Tested	Browse listings with search and category/price filters
Test Setup	Add sample listings to database: Mini Fridge (\$40, Appliances, ACTIVE), Desk Lamp (\$15, Dorm Essentials, ACTIVE), Refrigerator (\$120, Appliances, ACTIVE)
Action	Run the search/filter request
Expected result	Only Mini Fridge should be returned
Verification	Check API response JSON and confirm filtered results displayed correctly in UI

Component	Details
Feature Being Tested	Seller Creates a new listing
Test Setup	Title: Mini Fridge, Description: Used mini fridge in good condition, Price: 40, Category: Appliances, Status: ACTIVE
Action	Seller account already exists and is logged in
Expected result	New listing is created successfully and assigned to the seller
Verification	Check success response, verify listing appears in GET /listings, and confirm row is added to the Listings table in Postgres/pgAdmin

Component	Details
Feature Being Tested	Buyer sends a message to seller about a listing
Test Setup	Buyer, seller, and conversation thread already exist for a listing
Sample Input	"Hi, is this still available?"
Expected result	Message is stored and appears in the correct conversation
Verification	Check API response, verify message appears in chat UI, and confirm row is added to the Messages table

Thanks!

Do you have any questions?

Team 10

CREDITS: This presentation template was created by [Slidesgo](#), and includes icons, infographics & images by [Freepik](#)